NotesApp JWT Fix

Date: 30 October 2026

Class 7 lab, run against the local NotesApp clone. The app is served over HTTPS at `https://localhost` under the Class 5 configuration, so the commands use `curl -k` for the local self-signed certificate.

Predictions

Recorded before any change, as the lab requires:

Question	Prediction	Result
After all three fixes, will the old forged tokens still work?	**No**	Confirmed: both returned `unauthorized` (HTTP 401).
Will a normal login still work?	**Yes**	Confirmed: alice logged in and read her own note.

Audit line

```
LOGIN FIX — NotesApp flaw #4 (auth) closed
  Fix 1:  verify_jwt rejects alg:none and pins algorithm to HS256
  Fix 2:  JWT_SECRET is long + random, read from the environment (no s3cr3t, no default)
  Proof:  alg:none token        -> unauthorized (HTTP 401)
          s3cr3t-signed token   -> unauthorized (HTTP 401)
          real alice login      -> works (HTTP 302, dashboard HTTP 200, own note HTTP 200)
```

What changed

Fix 1 — reject `alg:none` and pin the algorithm to HS256

`verify_jwt` now requires the header algorithm to be exactly `HS256`:

```
    # The SERVER decides the algorithm. The token's header does not get to choose.
    # Anything that is not exactly HS256 (including "none") is rejected.
    if header.get("alg") != "HS256":
        return None
    signing_input = f"{h_b64}.{p_b64}".encode()
    expected = _b64url(hmac.new(JWT_SECRET.encode(), signing_input, hashlib.sha256).digest())
    if not hmac.compare_digest(expected, sig_b64):
        return None
    return payload
```

Every other value, including `none`, is rejected. The token header no longer chooses how the server verifies it.

Fix 2 — long random secret from the environment

`web/app.py` now reads a required environment variable with no fallback:

```
JWT_SECRET = os.environ["JWT_SECRET"]  # required; no weak default
```

`docker-compose.yml` passes a newly generated `secrets.token_urlsafe(32)` value to the `web` service, replacing `s3cr3t`. Removing the default means the app refuses to start if `JWT_SECRET` is missing, instead of silently falling back to a guessable value.

Fix 3 — real login still works

Alice logged in with a fresh session. The server issued a genuine token signed with the new secret: her dashboard loaded with her grades (16.2 and 10.5), and the token read her own note through `/api/notes/1` (HTTP 200).

Evidence

Both forged tokens from Class 6, replayed against the fixed app:

```
$ curl -k -H "Authorization: Bearer <alg:none token>" https://localhost/api/notes/3
{
  "error": "unauthorized"
}
HTTP 401

$ curl -k -H "Authorization: Bearer <s3cr3t-signed token>" https://localhost/api/notes/3
{
  "error": "unauthorized"
}
HTTP 401
```

Raw output: proof-forged-tokens.txt

Real alice login and the server-issued token:

```
$ curl -k -s -c <cookie jar> -X POST https://localhost/login -d "username=alice" -d
"password=alicepw" -w "HTTP %{http_code}\n"
HTTP 302
$ curl -k -s -b <cookie jar> -w "HTTP %{http_code}\n" https://localhost/dashboard
HTTP 200
$ grep "<h1>My grades</h1>" ...
  <h1>My grades</h1>
        <td>10.5</td>
        <td>16.2</td>
$ curl -k -s -H "Authorization: Bearer <genuine alice token>" https://localhost/api/notes/1
{"id": 1, "student": "alice", "grade": 16.2, ...}
HTTP 200
```

Raw output: proof-real-login.txt

The existing test suite also passes (`7 tests`, `OK`).

Secret handling

For this lab, the generated value is committed in `docker-compose.yml`, as the course instructs. In a real deployment the secret comes from a secrets store, never from a committed file.

Class 7 lab